UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RAFAELA

- Tecnicatura Universitaria en Programación -

Gestión de Desarrollo de Software



Unidad II: Administración de Proyectos de Software

Docente: Ing. Darío Haspert

Curso: 2º año - 2º cuatr.

< agosto de 2026 >

Índice

UNIDAD 2. Administración de Proyectos de Software 3
1. El espectro administrativo 3
2. Las personas 3
2.1. Los participantes (Stakeholders) 3
2.2. Líderes de equipo 4
2.3. El equipo de software 4
2.4. Problemas de coordinación y comunicación 5
3. El producto 5
3.1. Alcance y descomposición del software 5
4. El proceso 6
4.1. Integración del producto y el proceso 6
4.2. Descomposición del proceso 7
5. El proyecto 8
5.1. Inclusiones del Alcance 9
5.2. Exclusiones del Alcance 10
5.3. Supuestos del Proyecto 11
5.4. Restricciones 13
5.5. Criterios de Aceptación 15
5.6. Registro de Interesados (Stakeholders) 17
6. La Propuesta de Proyecto 20
6.1. Definición y propósito 20
6.2. Relación con el PMBOK 20
6.3. Estructura y componentes 21
6.4. La Propuesta de Proyecto en contextos ágiles 22
6.5. Errores frecuentes al redactar una Propuesta de Proyecto 22
7. El principio W^5HH 23
8. Resumen 23
Referencias Bibliográficas 24

UNIDAD 2. Administración de Proyectos de Software

La administración de proyectos de software organiza el trabajo desde la idea inicial hasta la entrega del producto. Implica coordinar personas, definir el alcance del producto, elegir un proceso adecuado y planificar el proyecto. Su objetivo es lograr un software de calidad, dentro del plazo y del presupuesto previstos.

1. El espectro administrativo

La administración efectiva de un proyecto de software se organiza en torno a cuatro aspectos (4 P): personas, producto, proceso y proyecto.

- El éxito de un proyecto de software depende en gran medida de las **personas**: su capacitación, coordinación, motivación y trabajo en equipo.
- Antes de planificar un proyecto, deben definirse los objetivos y el alcance del **producto**, considerar alternativas y reconocer las restricciones técnicas y administrativas. Esto permite estimar costos, tiempos y riesgos con mayor precisión.
- El **proceso** de software brinda el marco para organizar el desarrollo. Incluye actividades comunes a todos los proyectos y otras actividades transversales, como la calidad, la configuración y las mediciones.
- Los **proyectos** de software se planifican y controlan para manejar la complejidad. Una buena administración ayuda a prevenir problemas y a mantener el proyecto dentro de los objetivos previstos.

Estas cuatro dimensiones — personas, producto, proceso y proyecto — constituyen el marco organizador de esta unidad y se desarrollan en detalle en las secciones siguientes.

2. Las personas

El software lo crean las personas, y los proyectos tienen éxito cuando participan equipos bien formados y motivados. En esta sección se analiza quiénes intervienen en el desarrollo de software y cómo se organizan para lograr una ingeniería de software eficaz.

2.1. Los participantes (Stakeholders)

En todo proyecto de software participan distintas personas o grupos con intereses y responsabilidades diferentes. Entre ellos se encuentran:

- los **directivos o patrocinadores (sponsors)**, que impulsan y apoyan el proyecto;
- el **gerente de proyecto (PM: Project Manager)**, que planifica y coordina el trabajo;
- el **equipo técnico**, que desarrolla la solución;
- los **clientes**, que definen necesidades y requisitos; y
- los **usuarios finales**, que utilizan el sistema y aportan retroalimentación.

Para que el proyecto sea eficaz, es necesario organizar a estos participantes de modo que se aprovechen al máximo sus capacidades.

2.2. Líderes de equipo

La administración de proyectos de software exige habilidades técnicas y de liderazgo. Un líder de equipo debe coordinar personas, tomar decisiones y favorecer el trabajo colaborativo. Entre sus prácticas principales se encuentran dar el ejemplo, inspirar y compartir una visión clara, mejorar el proceso, dar autonomía al equipo y reconocer los logros. Además, debe centrarse en comprender el problema que hay que resolver, coordinar el flujo de ideas de las partes interesadas y valorar las contribuciones de todos los miembros del equipo.

2.3. El equipo de software

La organización del equipo de software depende de múltiples variables: la complejidad del proyecto, el tamaño del sistema, la experiencia de los integrantes y la necesidad de comunicación. No existe una estructura única válida para todos los proyectos, pero sí existen criterios que permiten tomar mejores decisiones al momento de conformar y organizar un equipo.

Mantei identifica siete factores del proyecto que deben considerarse al planificar la estructura del equipo de desarrollo:

1. La dificultad del problema que hay que resolver.
2. El tamaño del programa resultante, medido en líneas de código o puntos de función.
3. El tiempo que el equipo permanecerá unido durante el proyecto.
4. El grado en que el problema puede descomponerse en módulos independientes.
5. La calidad y confiabilidad requeridas para el sistema.
6. La rigidez de la fecha de entrega.
7. El grado de comunicación y colaboración requerido entre los integrantes.

Estos factores no determinan una estructura óptima universal, sino que orientan al responsable del proyecto para adaptar la organización del equipo al contexto específico de cada desarrollo.

Un equipo cohesionado trabaja mejor, con mayor motivación y productividad. DeMarco y Lister describen este fenómeno en su libro *Peopleware*: un equipo que funciona como un todo es un grupo tan fuertemente unido que el conjunto supera la suma de sus partes. Una vez que un equipo alcanza ese nivel de cohesión, su probabilidad de éxito aumenta considerablemente.

Sin embargo, no todos los equipos logran cohesionarse. Jackman identifica cinco factores que generan lo que denomina *toxicidad del equipo* — un entorno que destruye la colaboración y compromete el proyecto:

1. Un ambiente de trabajo frenético, con presión constante y cambios permanentes.
2. Frustración acumulada que genera fricciones entre los miembros del equipo.
3. Un proceso de software fragmentado o mal coordinado.
4. Roles y responsabilidades poco claros dentro del equipo.
5. Exposición continua y repetida al fracaso, sin mecanismos de corrección.

Conocer estos factores permite al responsable del proyecto actuar de manera preventiva: asegurarse de que el equipo tenga acceso a la información necesaria, que los objetivos no cambien innecesariamente, que los roles estén definidos con claridad y que existan mecanismos de retroalimentación y resolución de problemas.

En cuanto a la organización interna, los enfoques ágiles favorecen equipos pequeños, altamente motivados y autoorganizados. En Scrum, por ejemplo, el equipo tiene autonomía significativa para tomar decisiones técnicas y de gestión, se coordina mediante reuniones diarias y adapta su enfoque de forma continua a medida que avanza el proyecto.

2.4. Problemas de coordinación y comunicación

Los proyectos de software suelen presentar problemas de coordinación debido a su complejidad, a los cambios frecuentes y a la necesidad de interoperar con otros sistemas. Para enfrentarlos, el equipo debe contar con mecanismos de comunicación formal e informal entre los miembros del equipo y entre los distintos equipos. La comunicación formal incluye comunicación escrita, reuniones estructuradas y otros canales impersonales; la informal surge en el trabajo cotidiano, cuando los miembros del equipo consultan, comparten ideas y resuelven problemas directamente.

3. El producto

Un gerente de proyecto de software se enfrenta con un dilema en el comienzo mismo de un proyecto de software. Se requieren estimaciones cuantitativas y un plan organizado, pero no hay información sólida disponible. Un análisis detallado de los requerimientos del software proporcionaría la información necesaria para las estimaciones, pero el análisis usualmente tarda semanas o incluso meses en completarse. Además, los requerimientos pueden ser fluidos y cambiar con regularidad conforme avanza el proyecto.

Por eso, antes de estimar tiempos y costos, conviene analizar el producto y acotar su alcance.

3.1. Alcance y descomposición del software

La primera actividad en la administración del proyecto es determinar el alcance del software: qué contexto empresarial tendrá, qué datos procesará y qué funciones cumplirá. Su definición completa — incluyendo inclusiones, exclusiones, supuestos y restricciones — se desarrolla en la sección 5.

La definición del alcance debe ser clara y verificable, comprensible tanto a nivel directivo como técnico, e incluir datos cuantitativos cuando sea posible: número de usuarios simultáneos, tiempo de respuesta máximo, volumen de datos esperado; restricciones técnicas o administrativas y cualquier otra condición relevante para la planificación. Esta precisión es la que permite pasar de una descripción general del producto a estimaciones confiables de tiempo, costo y recursos.

Una vez delimitado el alcance, el trabajo se descompone en partes más pequeñas y manejables para facilitar la estimación y la planificación. Esta descomposición puede expresarse mediante una lista de funciones, casos de uso o, en entornos ágiles, historias de usuario. La descomposición no intenta ser

exhaustiva en esta etapa — eso ocurre durante la planificación detallada — sino proporcionar una estructura inicial sobre la cual trabajar.

4. El proceso

Las actividades marco que caracterizan el proceso de desarrollo de software son aplicables a todos los proyectos de software. La cuestión es seleccionar el modelo de proceso adecuado para el software que va a desarrollar el equipo de proyecto.

El equipo debe decidir qué modelo de proceso es el más adecuado para:

(1) los clientes que han solicitado el producto y las personas que realizarán el trabajo,
(2) las características del propio producto, y
(3) el entorno del proyecto en el que trabaja el equipo de software.

Una vez seleccionado un modelo de proceso, el equipo define un plan de proyecto preliminar basado en el conjunto de actividades del marco de procesos. Una vez establecido el plan preliminar, comienza la descomposición del proceso. Es decir, debe crearse un plan completo que refleje las tareas de trabajo necesarias para completar las actividades del marco.

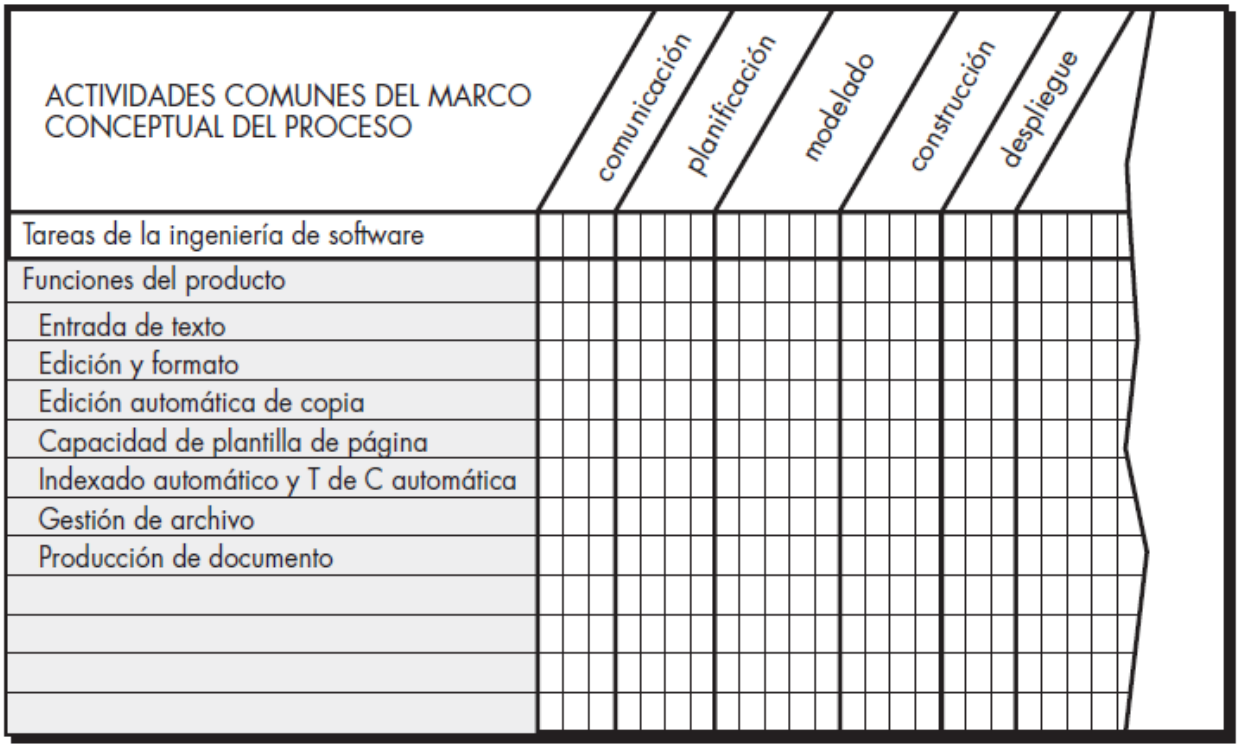
4.1. Integración del producto y el proceso

La planificación del proyecto comienza con la integración del producto y el proceso. Cada función que vaya a diseñar el equipo debe pasar por el conjunto de actividades del marco que se han definido para la organización del software.

El marco de procesos establece un esqueleto para la planificación del proyecto. Se adapta asignando un conjunto de tareas adecuado al proyecto. Supongamos que la organización ha adoptado las actividades genéricas del marco (comunicación, planificación, modelado, construcción e implementación).

Los miembros del equipo que trabajan sobre una función del producto aplicarán cada una de las actividades del marco a la misma. En esencia, se crea una matriz similar a la que se muestra en la siguiente figura. Cada función principal del producto (la figura enumera las funciones del software de procesamiento de palabras) o historia de usuario aparece en la columna de la izquierda. Las actividades del marco se enumeran en la fila superior.

Las tareas de ingeniería de software (para cada actividad del marco) se introducirían en la fila siguiente. La labor del gerente del proyecto (y de otros miembros del equipo) consiste en estimar los recursos necesarios para cada celda de la matriz, las fechas de inicio y finalización de las tareas asociadas a cada celda, y los productos de trabajo que se generarán como resultado de cada tarea.



4.2. Descomposición del proceso

Un equipo de software debe disponer de un grado significativo de flexibilidad a la hora de elegir el modelo de proceso de software más adecuado para el proyecto y las tareas de ingeniería de software que conforman dicho modelo una vez elegido. Un proyecto relativamente pequeño, similar a otros anteriores, podría llevarse a cabo de la mejor manera utilizando un enfoque de un solo sprint. Si el plazo es tan ajustado que no es razonable entregar la funcionalidad completa, una estrategia incremental podría ser la mejor opción. Del mismo modo, los proyectos con otras características (por ejemplo, requisitos inciertos, tecnología innovadora, clientes difíciles o potencial para una reutilización significativa de componentes) llevarán a la selección de otros modelos de proceso.

Una vez elegido el modelo de proceso, el marco conceptual del proceso se adapta a él. En todo caso, puede usarse el marco conceptual genérico de proceso que se estudió anteriormente. Funcionará para los modelos lineales, para modelos iterativos e incrementales, para modelos evolutivos e incluso para modelos concurrentes o de ensamble de componentes. El marco conceptual del proceso es invariante y sirve como la base para todo el trabajo que realiza una organización de software.

Pero las tareas del trabajo real sí varían. La descomposición del proceso comienza cuando el gerente de proyecto pregunta: ¿cómo logramos esta actividad del marco conceptual? Por ejemplo, un proyecto simple y relativamente pequeño puede requerir las siguientes tareas para la actividad de comunicación:

1. Desarrollar lista de clarificación de conflictos.
2. Reunirse con los participantes para abordar la clarificación de conflictos.
3. Desarrollar en conjunto una declaración del alcance, listando historias de usuario.
4. Revisar la declaración del alcance con todos los interesados y determinar la importancia de cada historia de usuario.
5. Modificar la declaración del alcance según se requiera.

Estos eventos representan una descomposición de proceso que es adecuada para el pequeño proyecto relativamente simple.

Ahora, considere un proyecto más complejo, que tenga un ámbito más amplio e impacto empresarial más significativo. Tal proyecto puede requerir las siguientes tareas para la **comunicación**:

1. Revisar la solicitud del cliente.
2. Planificar y programar una reunión formal, moderada, con todas las partes interesadas.
3. Realizar una investigación para especificar la solución propuesta y los enfoques existentes.
4. Preparar un «documento de trabajo» y una agenda para la reunión formal.
5. Llevar a cabo la reunión.
6. Desarrollar conjuntamente mini-especificaciones que reflejen las características de datos, funciones y comportamiento del software. Esto suele hacerse mediante el desarrollo de casos de uso que describan el software desde el punto de vista del usuario.
7. Revisar cada mini-especificación o caso de uso para verificar su corrección, coherencia y ausencia de ambigüedad.
8. Reunir las mini-especificaciones en un documento de alcance.
9. Revisar el conjunto de casos de uso con todos los interesados y determinar su importancia relativa para todas las partes interesadas.
10. Modificar el documento de alcance o los casos de uso según sea necesario.

Ambos proyectos realizan la actividad de marco conceptual que se llama **comunicación**, pero el primer equipo de proyecto realiza la mitad de tareas de trabajo de ingeniería de software que el segundo.

5. El proyecto

Para gestionar con éxito un proyecto de software, hay que comprender qué puede salir mal, de modo que se puedan evitar los problemas.

Las siguientes son características que están presentes en los proyectos de software exitosos y que también se encuentran en la mayoría de los modelos de proceso bien diseñados.

1. Requisitos claros y bien entendidos, aceptados por todas las partes interesadas
2. Participación activa y continua de los usuarios a lo largo de todo el proceso de desarrollo
3. Un gerente de proyecto con las habilidades de liderazgo necesarias, capaz de compartir la visión del proyecto con el equipo
4. Un plan y un calendario del proyecto elaborados con la participación de las partes interesadas para alcanzar los objetivos de los usuarios
5. Miembros del equipo cualificados y comprometidos
6. Miembros del equipo de desarrollo con personalidades compatibles que disfruten trabajando en un entorno colaborativo
7. Estimaciones realistas del calendario y el presupuesto que se supervisan y mantienen
8. Necesidades de los clientes que se comprenden y satisfacen
9. Miembros del equipo que experimentan un alto grado de satisfacción laboral
10. Un producto funcional que refleje el alcance y la calidad deseados

5.1. Inclusiones del Alcance

Las inclusiones del alcance son todas las funcionalidades, características, entregables y actividades que el proyecto de desarrollo de software se compromete a realizar y entregar. En otras palabras, expresan lo que sí formará parte del proyecto y permiten delimitar con claridad el trabajo esperado.

Definir las inclusiones resulta fundamental porque ayuda a alinear expectativas con el cliente, asignar recursos de manera adecuada, estimar costos y tiempos con mayor precisión y evitar confusiones sobre responsabilidades. En proyectos de software, esta delimitación suele abarcar funcionalidades del sistema, módulos o componentes, integraciones con otros sistemas, entregables técnicos, documentación de usuario, capacitación y, en algunos casos, soporte posterior a la implementación.

Componentes de las inclusiones:

Tipo	Descripción	Ejemplos
Funcionalidades	Qué hace el sistema.	El sistema registra incidentes; genera reportes automáticos.
Módulos o componentes	Partes internas del software.	Módulo de autenticación; módulo de facturación; API REST.
Integraciones	Conexiones con otros sistemas.	Integración con SAP; conexión con PostgreSQL.
Entregables técnicos	Artefactos producidos por el proyecto.	Código fuente; diagramas UML; manual técnico; pruebas unitarias.
Entregables de usuario	Materiales para el usuario final.	Manual de usuario; videos de capacitación; preguntas frecuentes.
Capacitación	Formación brindada a los usuarios o al personal.	Capacitación a 20 usuarios; tres sesiones de entrenamiento.
Mantenimiento	Soporte o asistencia posterior a la entrega.	Mantenimiento por 6 meses; atención de errores durante 30 días.

Ejemplos de declaración de inclusiones:

Un proyecto podría establecer, por ejemplo:

- Desarrollo de cinco módulos: autenticación, registro de incidentes, gestión de usuarios, reportes y notificaciones.
- Integración con el sistema de facturación existente mediante API REST.
- Base de datos PostgreSQL con 10 tablas.
- Capacidad para 100 usuarios simultáneos.
- Manual de usuario de 30 páginas.
- Manual técnico de 50 páginas.
- Capacitación a 20 usuarios en tres sesiones de dos horas.
- Seis meses de mantenimiento posterior a la implementación.

Criterios para formular inclusiones

Para que las inclusiones sean útiles en la gestión del proyecto, deben redactarse de manera:

- Específica, sin ambigüedades.
- Alcanzable, de acuerdo con el tiempo y los recursos disponibles.
- Relevante, vinculada con los objetivos del negocio.
- Verificable, es decir, susceptible de validación o prueba.

En un proyecto de software, una inclusión bien formulada evita expectativas incorrectas y facilita la estimación, el control del alcance y la aceptación del producto.

5.2. Exclusiones del Alcance

Las exclusiones del alcance son todas las funcionalidades, características, entregables y actividades que el proyecto de desarrollo de software no realizará. En otras palabras, delimitan qué queda fuera del compromiso del proyecto y ayudan a fijar fronteras claras entre lo que se entregará y lo que no.

Declarar exclusiones es una práctica muy importante porque evita el crecimiento silencioso del alcance ("scope creep"), protege el presupuesto y el cronograma, y aclara desde el inicio qué elementos quedarán para una etapa posterior o para otro proyecto. En desarrollo de software, esta precisión es especialmente valiosa, porque muchas veces las expectativas del cliente tienden a expandirse si no se establecen límites explícitos.

Componentes de las exclusiones:

Tipo	Descripción	Ejemplos
Funcionalidades no incluidas	Qué no hará el sistema.	No incluye chat en vivo; no incluye integración con Salesforce.
Módulos excluidos	Partes que no se desarrollarán.	No se incluye el módulo de pagos.
Integraciones no realizadas	Sistemas con los que no habrá conexión.	No se integra con un sistema legacy de contabilidad.
Responsabilidades ajenas al equipo	Actividades fuera del trabajo del proyecto.	No incluye migración de datos; no incluye provisión de hardware.
Mejoras futuras	Funcionalidades previstas para otra etapa.	La mejora de UX se abordará en el proyecto 2.
Limitaciones técnicas explícitas	Alcances que no se garantizan.	No se garantiza soporte para Windows 7.

Ejemplo de declaración de exclusiones:

Un proyecto podría declarar expresamente lo siguiente:

- No incluye el desarrollo del módulo de pagos.
- No incluye integración con el sistema *legacy* de contabilidad.
- No incluye la migración de datos históricos.
- No incluye la provisión de hardware ni de servidores.
- No incluye chat en vivo.
- No incluye soporte para Windows 7 o versiones anteriores.
- No incluye capacitación adicional para más de 20 personas.
- No incluye mantenimiento posterior a los 6 meses acordados.

Exclusión y restricción:

Aunque a veces se parecen, no son lo mismo. La exclusión define algo que queda fuera del alcance del proyecto; la restricción, en cambio, fija un límite que condiciona la ejecución del trabajo.

Aspecto	Exclusión	Restricción
Sentido	Algo que no se hará.	Algo que debe respetarse.
Relación con el alcance	Queda fuera del proyecto.	Condiciona la ejecución.
Posibilidad de cambio	Puede incorporarse en otro proyecto.	No puede modificarse sin replantear el proyecto.
Ejemplo	No se desarrollará el módulo de pagos.	El sistema debe usar Java 17.

Cómo identificar exclusiones

Para definirlas con claridad, suele ser útil revisar:

1. Qué espera el cliente pero no pidió explícitamente.
2. Qué requerimientos se mencionaron pero no podrán implementarse.
3. Qué partes del sistema quedan fuera del objetivo principal.
4. Qué funcionalidades podrían generar expansión del alcance en el futuro.

La redacción de exclusiones ayuda a controlar el alcance y a evitar que el proyecto se desborde por agregados informales o supuestos no acordados.

5.3. Supuestos del Proyecto

Los supuestos del proyecto son condiciones, hechos o afirmaciones que se aceptan como verdaderos al momento de planificar, aunque no exista certeza total sobre su cumplimiento. En gestión de proyectos, formular supuestos es necesario porque la planificación siempre se realiza con información incompleta; por eso, el equipo debe establecer ciertas condiciones de trabajo para poder avanzar con la definición del alcance, la estimación y la programación.

A diferencia de las restricciones, los supuestos no constituyen límites obligatorios del proyecto, sino hipótesis de planificación. Sin embargo, también implican riesgo: si un supuesto no se cumple, pueden verse afectados el cronograma, el costo, el alcance o incluso la viabilidad técnica de la solución.

Características clave de los supuestos:

- Se aceptan como válidos para poder planificar.
- No están garantizados.
- Implican riesgo si resultan falsos.
- Deben registrarse explícitamente.
- Deben revisarse durante el desarrollo del proyecto.

Supuesto vs. Restricción:

Aspecto	Supuesto	Restricción
Naturaleza	Hipótesis de trabajo.	Límite obligatorio.
Grado de certeza	Puede no cumplirse.	Debe cumplirse.
Efecto del cambio	Afecta la planificación.	Exige replanificación.
Ejemplo	El cliente responderá en 2 semanas.	El presupuesto máximo es de 500.000 dólares.

Tipos de supuestos:

Tipo	Descripción	Ejemplos
Recursos	Disponibilidad de personal, herramientas o infraestructura.	El equipo tendrá tres desarrolladores full-time.
Tecnología	Funcionamiento y disponibilidad de la tecnología prevista.	Azure estará disponible sin interrupciones.
Requerimientos	Estabilidad esperada de los requisitos.	Los requisitos no cambiarán significativamente.
Entorno	Condiciones externas favorables.	No habrá cambios regulatorios durante el proyecto.
Stakeholders	Participación de los involucrados clave.	El cliente participará en reuniones semanales.
Tiempo	Cumplimiento probable del calendario.	El proyecto se completará en 12 meses.
Costo	Suficiencia del presupuesto asignado.	El presupuesto de 500.000 dólares será suficiente.

Datos	Disponibilidad de información para análisis o migración.	Los datos históricos estarán disponibles.
Integración	Disponibilidad de interfaces o servicios externos.	SAP tendrá una API REST disponible.
Usuarios	Disponibilidad de usuarios para pruebas o capacitación.	Habrá 20 usuarios disponibles para capacitación.

Gestión de supuestos

Los supuestos se identifican al inicio, se documentan y se revisan durante la ejecución, porque pueden convertirse en una fuente de riesgo si dejan de ser válidos. En términos prácticos, esto suele plasmarse en un **registro de supuestos**, donde se anotan el supuesto, su tipo, el riesgo asociado, la fecha de validación y su estado.

5.4. Restricciones

Las restricciones son límites o condiciones que enmarcan la planificación y la ejecución de un proyecto de desarrollo de software. A diferencia de los supuestos, que son hipótesis aceptadas para poder planificar, las restricciones constituyen exigencias reales que no pueden soslayarse sin afectar la viabilidad del proyecto. En la práctica, definen el margen de decisión del equipo y obligan a adaptar el alcance, los recursos y la estrategia de trabajo a un contexto concreto.

En la literatura de gestión de proyectos, las restricciones más conocidas son las del **triángulo de hierro**: **alcance, tiempo y costo**. Estos tres elementos están estrechamente vinculados, de modo que modificar uno de ellos suele repercutir en los otros dos. En proyectos de software, además, suele incorporarse explícitamente la **calidad** y, en muchos casos, la disponibilidad de **recursos** como restricciones centrales del proyecto.

Si el plazo se acorta, generalmente será necesario aumentar recursos o elevar el costo. Si se reduce el presupuesto, puede requerirse más tiempo o una disminución del alcance. Y si se acota el alcance, es posible mantener el tiempo y el costo dentro de límites más realistas. La calidad también forma parte de este equilibrio: cuando se reduce excesivamente el tiempo o el costo, o cuando se disminuye el alcance de manera forzada, la calidad del producto puede verse comprometida.

Tipo	Descripción	Ejemplos
Tiempo	Plazos, fechas límite o duración máxima del proyecto.	El sistema debe entregarse antes de fin de año; la primera versión debe estar lista en 90 días.
Costo	Presupuesto máximo disponible para el proyecto.	El proyecto no puede superar 500.000 dólares.
Alcance	Límites sobre funcionalidades, módulos o entregables.	Solo se desarrollará la versión básica; el módulo de pagos queda fuera de esta etapa.

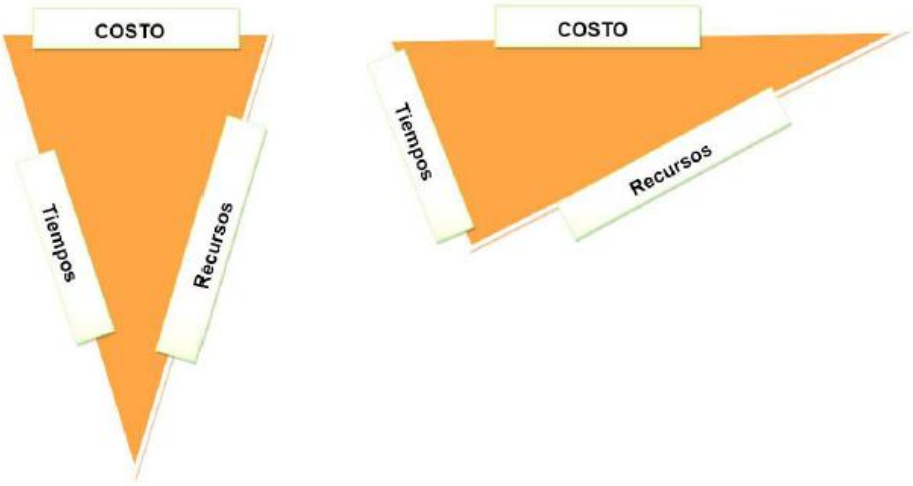
Calidad	Nivel mínimo aceptable de desempeño, confiabilidad o seguridad.	El sistema debe responder en menos de 3 segundos; debe cumplir con estándares de seguridad.
Recursos	Disponibilidad de personas, equipos, licencias o infraestructura.	Solo habrá dos desarrolladores asignados; el equipo contará con un único servidor de pruebas.
Técnicas	Condiciones impuestas por la arquitectura, la plataforma o la compatibilidad.	El sistema debe desarrollarse en Java 17; debe integrarse con PostgreSQL.
Legales o institucionales	Normas, reglamentos o políticas organizacionales.	Debe cumplir con la normativa de protección de datos; debe utilizar herramientas aprobadas por la empresa.

El triángulo de hierro

El triángulo de hierro expresa una relación de tensión entre alcance, tiempo y costo. En este modelo, si se intenta modificar uno de los lados sin ajustar los otros, se altera necesariamente el equilibrio del proyecto. En desarrollo de software, esto significa que no es razonable esperar más funcionalidades, en menos tiempo y con menor presupuesto, sin afectar la calidad o sin introducir riesgos adicionales.



Cambio principal	Efecto habitual
Reducir tiempo	Aumenta el costo o exige más recursos.
Reducir costo	Aumenta el tiempo u obliga a reducir el alcance.
Aumentar alcance	Requiere más tiempo, más costo o más recursos.
Aumentar calidad	Suele demandar más tiempo, más costo o más recursos.



Ejemplos de restricciones en proyectos de software

- Un sistema para una clínica debe estar listo antes de la inauguración de una nueva sede (**tiempo**).
- El presupuesto aprobado no puede superar un monto determinado (**costo**).
- Solo se implementará la gestión básica de turnos y no la historia clínica completa (**alcance**).
- El software debe responder en menos de tres segundos y operar con alta confiabilidad (**calidad**).
- El proyecto contará con un equipo reducido y una infraestructura limitada (**recursos**).
- La solución deberá desarrollarse sobre una plataforma tecnológica ya definida por la organización (**técnica**).
- El producto tendrá que ajustarse a una ley específica de protección de datos (**legal/institucional**).

Relación con los supuestos

Conviene distinguir estas restricciones de los supuestos. Los supuestos son condiciones que se aceptan como verdaderas para planificar; las restricciones, en cambio, son límites efectivos que condicionan la ejecución. En el trabajo del proyecto, ambos deben identificarse desde el inicio, documentarse y revisarse, porque de ellos depende la factibilidad del plan.

5.5. Criterios de Aceptación

Los criterios de aceptación son las condiciones que un entregable de software debe cumplir para ser considerado válido y aceptado formalmente por el cliente, el sponsor o los interesados autorizados. Su función es objetivar la validación del resultado y establecer, desde el inicio del proyecto, qué requisitos, atributos de calidad o condiciones de desempeño deben verificarse antes de dar por concluido un entregable.

En la bibliografía de gestión de proyectos, estos criterios aparecen vinculados con la validación del alcance y con la aceptación formal de los entregables, es decir, con la comprobación de que lo construido coincide con lo acordado. En desarrollo de software, son especialmente útiles porque permiten convertir expectativas generales en condiciones verificables, medibles y comunicables.

Características de los criterios de aceptación

- Deben redactarse antes de iniciar el desarrollo del proyecto o entregable (durante la fase de planificación o refinamiento).
- Deben ser claros, específicos y sin ambigüedad.
- Deben poder verificarse mediante revisión, inspección o prueba.
- Deben permitir una decisión objetiva de aceptación o rechazo.
- Deben incluir, cuando corresponda, condiciones funcionales, de calidad y de rendimiento.

Criterios de aceptación y validación del alcance

Aspecto	Criterios de aceptación	Validación del alcance
Finalidad	Definir qué debe cumplir el entregable.	Confirmar formalmente que el entregable satisface lo acordado.
Momento de uso	Se fijan durante la planificación.	Se aplican al verificar la entrega.
Resultado	Criterio de evaluación.	Aceptación o solicitud de cambios.
Enfoque	Especificación verificable.	Acto formal de validación.

Ejemplos

En un proyecto de software, pueden establecerse criterios como los siguientes:

- El usuario debe poder registrarse en menos de tres pasos.
- El sistema debe generar un reporte mensual en formato PDF.
- El tiempo de respuesta no debe superar tres segundos.
- El administrador debe poder crear, editar y desactivar cuentas.
- La versión inicial debe aprobarse en pruebas de aceptación con el cliente.
- La integración con el sistema externo debe completar transacciones sin errores.

Relación con historias de usuario

En metodologías ágiles, los criterios de aceptación suelen escribirse para historias de usuario y ayudan a definir cuándo una historia está validada desde el punto de vista del negocio y lista para aceptarse. Sin embargo, su uso no se limita a lo ágil: también pueden aplicarse a módulos, versiones, prototipos o entregables completos del proyecto.

Importancia en proyectos de software

Los criterios de aceptación reducen la ambigüedad, alinean expectativas y facilitan la validación de entregables. En términos de gestión de proyectos, aportan un puente entre lo que se definió en el alcance y lo que se evalúa al momento de aceptar la entrega.

5.6. Registro de Interesados (Stakeholders)

El registro de interesados es el documento o listado donde se identifican las personas, grupos u organizaciones que tienen interés en el proyecto o pueden verse afectados por él. En proyectos de desarrollo de software, este registro es importante porque permite reconocer desde el inicio a los actores que intervienen en la definición del alcance, la validación de los entregables y la toma de decisiones.

El objetivo principal del registro de interesados es ordenar la relación entre el proyecto y sus actores. No todos los interesados tienen el mismo nivel de influencia ni el mismo grado de participación: algunos financian o patrocinan el proyecto, otros definen necesidades, otros usan el sistema y otros lo construyen o administran. Identificarlos con claridad facilita la comunicación, la coordinación y el control de expectativas.

Clasificación de interesados

Tipo de interesado	Rol en el proyecto	Ejemplos
Patrocinador	Impulsa o financia el proyecto.	Director general, gerente de área.
Cliente	Define necesidades y valida el alcance.	Área administrativa, responsable funcional.
Usuario final	Utiliza el sistema una vez implementado.	Operadores, administrativos, técnicos.
Equipo del proyecto	Diseña, desarrolla y prueba la solución.	Analista, programador, tester.
Interesado externo	Tiene influencia normativa o institucional.	Organismo regulador, proveedor de plataforma.

Ejemplos

En un sistema de gestión de turnos médicos, el registro de interesados podría incluir:

- Patrocinador: dirección de la clínica.
- Cliente operativo: recepción.
- Usuarios finales: pacientes, secretarias y médicos.
- Equipo del proyecto: analista, desarrolladores y testers.
- Interesado externo: autoridad de protección de datos.

En una aplicación de facturación, podrían intervenir:

- Patrocinador: gerencia comercial.
- Cliente: área de administración.
- Usuarios finales: personal de facturación y contabilidad.
- Equipo del proyecto: desarrolladores, analistas y soporte.
- Interesado externo: proveedor de servicios fiscales o entidad regulatoria.

Componentes del registro

Un registro de interesados suele incluir datos que permitan identificar al actor, entender su vínculo con el proyecto y definir cómo gestionarlo.

Componente	Descripción	Ejemplo
Nombre del interesado	Identificación de la persona o grupo.	Dirección de la clínica.
Tipo de interesado	Categoría o rol dentro del proyecto.	Patrocinador, cliente, usuario final.
Interés principal	Qué espera o necesita del proyecto.	Reducir tiempos de atención.
Nivel de influencia	Grado de poder para afectar el proyecto.	Alto, medio, bajo.
Nivel de participación	Intensidad con la que interviene.	Alta en validación, baja en desarrollo.
Necesidades de comunicación	Qué información requiere y con qué frecuencia.	Reportes semanales.
Observaciones	Datos relevantes para la gestión.	Requiere aprobación formal.

Matriz de poder e interés

La matriz de poder vs. interés clasifica a los interesados según dos variables: su capacidad de influencia sobre el proyecto y el grado de interés que tienen en él. Esto ayuda a decidir cómo tratarlos desde la gestión.

	Bajo interés	**Alto interés**
Alto poder	Mantener satisfecho	Gestionar de cerca
Bajo poder	Monitorear	Mantener informado

Los interesados con alto poder y alto interés requieren seguimiento cercano, porque pueden influir directamente en el rumbo del proyecto. En cambio, los de bajo poder y bajo interés suelen necesitar solo monitoreo ocasional.

Ejemplo

En un sistema de gestión de turnos médicos, el registro de interesados podría organizarse así:

Interesado	Tipo	Poder	Interés	Estrategia
Dirección de la clínica	Patrocinador	Alto	Alto	Gestionar de cerca.

Recepción	Cliente operativo	Medio	Alto	Mantener informada.
Médicos	Usuarios finales	Medio	Alto	Mantener informados.
Equipo de desarrollo	Equipo del proyecto	Alto	Alto	Gestionar de cerca.
Autoridad de datos	Externo	Alto	Bajo	Mantener satisfecho.

Este tipo de registro permite saber quién decide, quién usa el sistema y quién debe recibir información durante el proyecto.

Utilidad del registro

El registro de interesados permite:

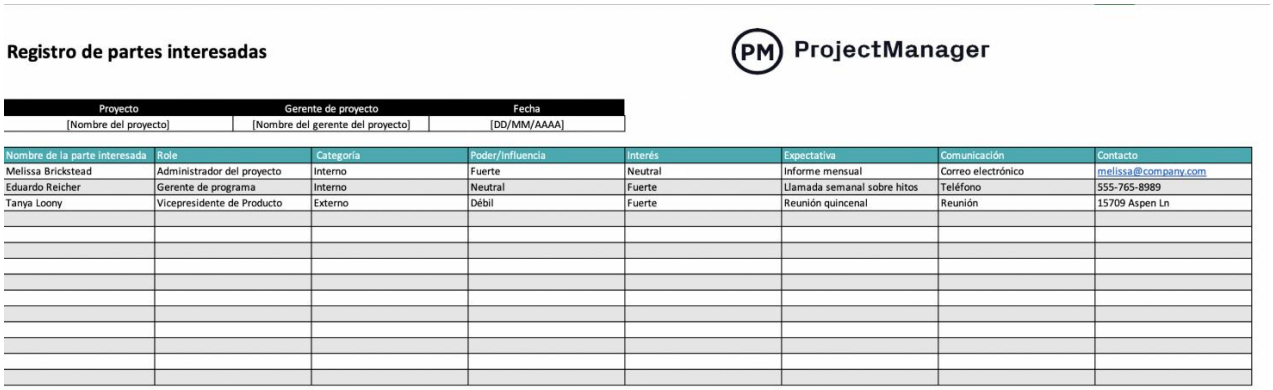
- identificar quién debe ser informado;
- reconocer quién toma decisiones;
- establecer a quién consultar durante el proyecto;
- anticipar conflictos de expectativas;
- organizar la comunicación según el nivel de influencia de cada actor.

Además, al vincularse con la matriz de poder e interés, el registro no solo lista actores, sino que también permite priorizarlos según su influencia real en el proyecto.

Relación con la gestión del proyecto

En desarrollo de software, los interesados no son un elemento accesorio: influyen en los requisitos, en las restricciones, en los criterios de aceptación y en la aprobación final del producto. Por eso, documentarlos desde el inicio ayuda a reducir malentendidos y a mejorar la conducción del proyecto.

Ejemplo de Registro de Interesados



Registro de partes interesadas

PM ProjectManager

Proyecto	Gerente de proyecto	Fecha
[Nombre del proyecto]	[Nombre del gerente del proyecto]	[DD/MM/AAAA]

Nombre de la parte interesada	Role	Categoría	Poder/influencia	Interés	Expectativa	Comunicación	Contacto
Melissa Brickstead	Administrador del proyecto	Interno	Fuerte	Neutral	Informe mensual	Correo electrónico	melissa@company.com
Eduardo Reicher	Gerente de programa	Interno	Neutral	Fuerte	Llamada semanal sobre hitos	Teléfono	555-765-8989
Tanya Loony	Vicepresidente de Producto	Externo	Débil	Fuerte	Reunión quincenal	Reunión	15709 Aspen Ln

Fuente: https://www.projectmanager.com/es/plantilla-de-registro-de-interesados

6. La Propuesta de Proyecto

Una vez que se han definido los elementos fundamentales del proyecto —alcance, interesados, supuestos, restricciones y criterios de aceptación— el equipo de gestión se encuentra en condiciones de formalizar toda esa información en un único documento de inicio: la **Propuesta de Proyecto**, conocida en la literatura como *Project Charter* o Acta de Constitución del Proyecto.

6.1. Definición y propósito

La Propuesta de Proyecto es un documento formal que autoriza la existencia de un proyecto y otorga al responsable de gestión la autoridad necesaria para utilizar los recursos de la organización en el logro de los objetivos documentados.

En términos prácticos, este documento cumple varias funciones simultáneas dentro de la organización:

- Reconoce formalmente la existencia del proyecto.
- Establece y comunica el propósito del proyecto a todos los interesados.
- Define los límites de autoridad del responsable del proyecto.
- Garantiza que los principales interesados acuerden formalmente la descripción del proyecto y se comprometan con su meta.
- Proporciona una vista de alto nivel del proyecto al documentar su objetivo, descripción y propósito.

En su esencia, el documento responde tres preguntas fundamentales: qué problema se está resolviendo, qué está fuera del alcance, y cómo se medirá el éxito.

6.2. Relación con el PMBOK

El estándar internacional de referencia para la gestión de proyectos, el *Project Management Body of Knowledge* (PMBOK®) del Project Management Institute (PMI), define la Propuesta de Proyecto como el artefacto de inicio del proceso de gestión. El *Project Charter* es identificado en el PMBOK® Guide, 7ª edición (PMI, 2021) como el documento que autoriza formalmente el proyecto y provee al responsable de gestión con la autoridad para aplicar recursos organizacionales a las actividades del proyecto.

Es importante distinguir este documento de otros que aparecen más adelante en el ciclo de vida:

Documento	Momento	Nivel de detalle
Propuesta de Proyecto (Project Charter)	Inicio	Alto nivel, sin detalles técnicos
Plan de Gestión del Proyecto	Planificación	Detallado, cubre todas las áreas
Enunciado del Alcance	Planificación	Descripción detallada del alcance

6.3. Estructura y componentes

Una propuesta de proyecto bien estructurada tiene entre tres y diez páginas, es de alto nivel y no contiene detalles técnicos exhaustivos; esos detalles aparecen en etapas posteriores de planificación.

Los componentes esenciales son los siguientes:

a) Título y datos generales: Nombre del proyecto, fecha de elaboración, versión del documento, nombre del equipo o responsable de gestión, y nombre del patrocinador o solicitante.

b) Descripción del proyecto: Exposición clara y concisa de qué se va a desarrollar. Responde a la pregunta *¿qué es el proyecto?* Debe incluir el contexto o problema que origina el proyecto, la solución propuesta en términos generales, y el tipo de producto o sistema a desarrollar.

Ejemplo: *"Se desarrollará un sistema web de gestión de turnos para una clínica médica, que permita a pacientes registrarse, solicitar turnos en línea y recibir notificaciones automáticas, y a los profesionales visualizar su agenda diaria."*

c) Objetivos del proyecto: Los objetivos deben identificarse y documentarse asegurando que sean específicos, medibles, alcanzables, relevantes y acotados en el tiempo (criterio SMART, por sus siglas en inglés: *Specific, Measurable, Achievable, Relevant, Time-bound*).

Un objetivo correctamente formulado según el criterio SMART tiene la siguiente estructura:

"[Verbo en infinitivo] + [resultado concreto] + [condición o restricción] + [plazo]"

Ejemplos de objetivos bien formulados:

- Desarrollar un módulo de registro de pacientes que permita el alta, modificación y baja de datos personales, en un plazo de cuatro semanas.
- Implementar un sistema de notificaciones por correo electrónico que confirme o recuerde turnos con al menos 24 horas de anticipación, antes de la entrega final del cuatrimestre.
- Generar reportes semanales de ocupación de agenda por profesional, exportables en formato PDF.

Se distinguen dos tipos de objetivos:

- **Objetivo general:** describe el propósito central del proyecto en términos amplios. Hay uno solo.
- **Objetivos específicos:** descomponen el objetivo general en resultados concretos y verificables. Suele haber entre tres y seis.

d) Alcance (Inclusiones y Exclusiones): Remite a lo desarrollado en las secciones 5.1 y 5.2 de esta unidad. En la Propuesta de Proyecto, el alcance se expresa de manera resumida pero explícita, evitando ambigüedades que puedan derivar en conflictos durante el desarrollo.

e) Supuestos y Restricciones: Remite a las secciones 5.3 y 5.4. Se listan brevemente en la propuesta para que todos los interesados los conozcan desde el inicio.

f) Criterios de Aceptación: Remite a la sección 5.5. Se incluyen al menos los criterios principales que permitirán determinar si el producto final es aceptable.

g) Registro de Interesados (resumen): Remite a la sección 5.6. En la propuesta se identifican al menos los interesados clave; el registro completo puede mantenerse como documento separado.

h) Entregables de alto nivel: Lista de los productos o resultados concretos que el proyecto producirá. No es una lista técnica detallada, sino una enumeración de los artefactos principales: por ejemplo, *sistema funcionando en ambiente de prueba*, *documentación técnica*, *manual de usuario*, *informe final*.

i) Cronograma de alto nivel / Hitos: Fechas o plazos estimados para los principales hitos del proyecto. No se trata de un Gantt detallado —eso corresponde a la planificación— sino de una línea de tiempo orientativa.

j) Recursos estimados: Descripción general del equipo de trabajo (cantidad de personas, roles previstos) y recursos tecnológicos necesarios (herramientas, infraestructura, etc.).

k) Firmas o aprobaciones: La propuesta es firmada por el patrocinador del proyecto, quien generalmente es un miembro de jerarquía con autoridad de decisión, y al firmarla autoriza formalmente el inicio del proyecto y otorga al responsable de gestión la autoridad para utilizar los recursos organizacionales. En el contexto académico, esta función la cumple el docente o la cátedra al aprobar la propuesta presentada por el grupo.

6.4. La Propuesta de Proyecto en contextos ágiles

En entornos de desarrollo ágil, la propuesta no desaparece sino que se adapta. Según el Agile Practice Guide del PMI, en contextos ágiles se promueven versiones más livianas del Project Charter que se alinean con principios de entrega continua y flexibilidad, permitiendo que el documento evolucione a medida que el equipo obtiene información real durante las iteraciones.

En la práctica, un equipo Scrum puede tener una propuesta inicial breve (a veces llamada *project brief* o *inception document*) que se complementa y refina durante la fase de *backlog refinement* de las primeras iteraciones.

6.5. Errores frecuentes al redactar una Propuesta de Proyecto

Entre los errores más comunes se encuentran: mezclar elementos de otros documentos con la propuesta, sobrecargar el documento con detalles que corresponden a etapas posteriores, no involucrar a los interesados en su elaboración, establecer objetivos vagos o no medibles, y omitir la autorización formal.

En el ámbito del desarrollo de software, se agregan errores específicos:

- Confundir objetivos del proyecto con funcionalidades del sistema (los objetivos son resultados de negocio, no características técnicas).
- No distinguir lo que *está* incluido de lo que *no está* incluido en el alcance.
- Redactar supuestos como si fueran hechos confirmados.
- Omitir criterios de aceptación, dejando la evaluación del éxito librada a interpretaciones subjetivas.

7. El principio W^{5}HH

En un excelente ensayo acerca del proceso de software y los proyectos, Barry Boehm [Boe96] afirma: "necesita un principio de organización que reduzca la escala a fin de proporcionar planes [de proyecto] simples para proyectos simples". Boehm sugiere un enfoque que aborda los objetivos del proyecto, hitos y calendarios, responsabilidades, enfoques administrativos y técnicos, y recursos requeridos. Él lo llama *principio W5HH*, por una serie de preguntas que conducen a una definición de las características clave del proyecto y al plan de proyecto resultante:

¿**Por qué (why)** se está desarrollando el sistema? Todas las partes interesadas deben evaluar la validez de los motivos empresariales que justifican el trabajo de desarrollo de software. ¿Justifica el objetivo empresarial el gasto en personal, tiempo y dinero?

¿**Qué (what)** se va a hacer? Se define el conjunto de tareas necesarias para el proyecto.

¿**Cuándo (when)** se hará? El equipo establece un calendario del proyecto determinando cuándo se llevarán a cabo las tareas del proyecto y cuándo se alcanzarán los hitos.

¿**Quién (who)** es responsable de cada función? Se definen el papel y la responsabilidad de cada miembro del equipo de software.

¿**Dónde (where)** se ubican dentro de la organización? No todos los roles y responsabilidades recaen en los profesionales del software. El cliente, los usuarios y otras partes interesadas también tienen responsabilidades.

¿**Cómo (how)** se llevará a cabo el trabajo desde el punto de vista técnico y de gestión? Una vez establecido el alcance del producto, se debe definir una estrategia técnica y de gestión para el proyecto.

¿**Cuánto (how much)** se necesita de cada recurso? La respuesta a esta pregunta se obtiene elaborando estimaciones basadas en las respuestas a las preguntas anteriores.

El principio W^{5}HH de Boehm es aplicable sin importar el tamaño o complejidad de un proyecto de software. Las preguntas indicadas ofrecen un excelente esquema de planificación.

8. Resumen

La administración de proyectos de software es una actividad sombrilla dentro de la ingeniería de software. Comienza antes de iniciar cualquier actividad técnica y continúa a lo largo del modelado, construcción y despliegue del software de cómputo.

Cuatro P tienen influencia sustancial sobre la administración del proyecto de software: personas, producto, proceso y proyecto. Las personas deben organizarse en equipos eficaces, motivados para hacer trabajo de software de alta calidad, y coordinarse para lograr comunicación efectiva. Los requerimientos del producto deben comunicarse de cliente a desarrollador, dividirse (descomponerse) en sus partes constitutivas y ubicarse para su trabajo por parte del equipo de software. El proceso debe

adaptarse al personal y al producto. Se selecciona un marco conceptual común al proceso, se aplica un paradigma de ingeniería de software adecuado y se elige un conjunto de tareas de trabajo para llevarlo a cabo. Finalmente, el proyecto debe organizarse de forma que permita triunfar al equipo de software.

El elemento esencial en todos los proyectos de software es el personal. Los ingenieros del software pueden organizarse en diferentes estructuras de equipo que van desde las jerarquías tradicionales de control hasta los equipos de "paradigma abierto". Para apoyar el trabajo del equipo, pueden aplicarse varias técnicas de coordinación y comunicación. En general, las revisiones técnicas y la comunicación informal persona a persona tienen más valor para los profesionales.

La actividad de administración del proyecto abarca medición y métricas, estimación y calendarización, análisis de riesgos, rastreo y control.

Referencias Bibliográficas



[1] *Ingeniería de Software: Un enfoque práctico*. 7ma Edición. Roger S. Pressman. McGrawHill. 2010.

[2] *Software Engineering: a practitioner's approach*. 9th Edition. Roger S. Pressman, Bruce R. Maxim. McGrawHill. 2020.

[3] Compilación de apuntes sobre conceptos fundamentales de la Ingeniería de Software - 2° Edición - Lic. César Ariel Briano

[4] Project Management Institute. (2021). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)* (7ª ed.). PMI.

[5] Project Management Institute. (2017). *Agile Practice Guide*. PMI.